

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	VIVETHA B	Reg. No.:	192424170
Date:		Duration:	60 minutes

Code of Conduct

I VIVETHA B 192424170 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: VIVETHA B

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

PHISHING EMAIL DETECTION AND CYBER THREAT INTELLIGENCE SYSTEM

- **Problem Overview**

- Explain the assigned problem statement and summarize the implemented solution.

Phishing is one of the most common forms of cyberattack in which attackers attempt to deceive users into revealing sensitive information such as usernames, passwords, banking details, authentication credentials, or personal information.

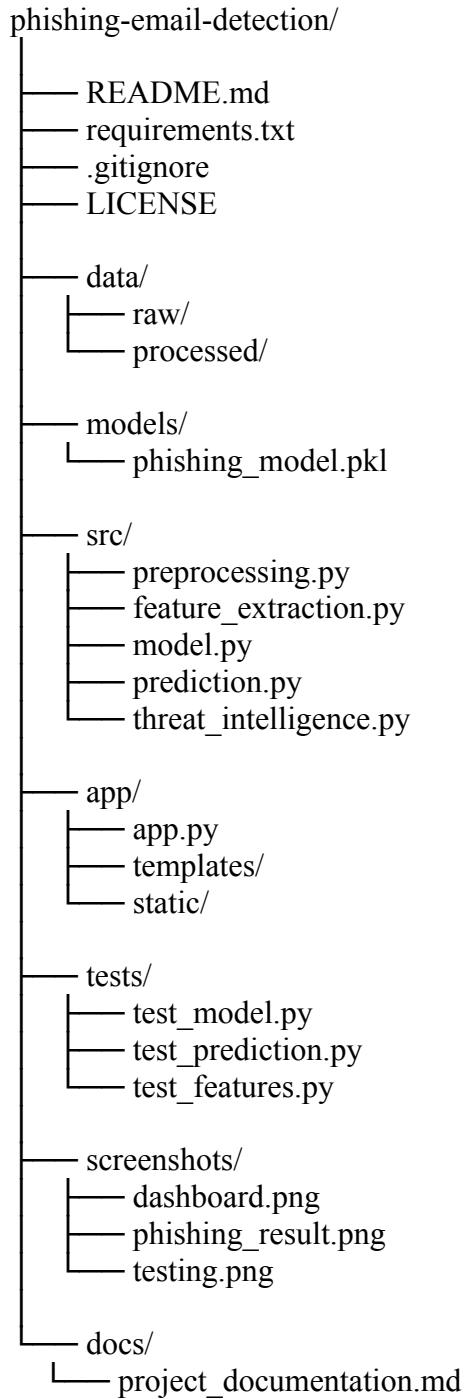
Phishing attacks are frequently delivered through emails that appear to originate from trusted organizations, financial institutions, educational institutions, companies, or individuals. These emails may contain malicious links, suspicious attachments, fake login pages, urgent requests, or misleading messages.

Traditional email filtering systems may fail to identify sophisticated phishing emails because attackers continuously modify their techniques. Therefore, an intelligent system capable of analyzing email content and identifying suspicious characteristics can help improve cybersecurity.

The **Phishing Email Detection and Cyber Threat Intelligence System** is designed to detect potentially malicious or phishing emails using automated analysis. The system examines email-related characteristics and generates a prediction indicating whether an email is legitimate or suspicious.

- Describe the project objectives and key functionalities.
 - To develop an automated phishing email detection system.
 - To identify suspicious characteristics present in email messages.
 - To classify emails as legitimate or phishing.
 - To reduce the possibility of users interacting with malicious emails.
 - To provide useful cyber threat intelligence.
 - To improve the awareness of suspicious URLs and email content.
 - To provide a simple and user-friendly interface.
 - To validate the system using appropriate test cases.
 - To maintain the source code using Git and GitHub.
 - To follow good software engineering practices involving documentation, testing, version control, and maintainability.
- **Repository Organization**
 - Evaluate the repository structure, folder organization, and file naming conventions.

A well-organized repository for the project can follow the structure below:



The actual repository structure should be compared with this recommended organization.

- Justify how the repository layout supports maintainability and collaboration.

A structured repository improves software development in several ways.

Maintainability

Developers can quickly locate:

- Source code
- Dataset
- Models
- Tests
- Documentation
- Configuration files

Collaboration

Team members can independently work on different components.

For example:

- Developer 1 → Machine learning model
- Developer 2 → Frontend
- Developer 3 → Threat intelligence
- Developer 4 → Testing and documentation

Git branches can then be used to combine these changes safely.

- **Code Quality Review**
 - Review the source code for readability, modularity, coding standards, and documentation.

The source code should be easy to understand.

Good practices include:

- Meaningful variable names
- Proper indentation
- Small functions
- Comments for complex operations
- Avoiding unnecessary duplicate code
- Consistent formatting

Example:

```
email_text = preprocess_email(email)
features = extract_features(email_text)
prediction = model.predict(features)
```

This is easier to understand than using unclear variable names such as:

```
x = p(e)
y = f(x)
z = m(y)
```

Modularity

The project should divide functionality into separate modules.

For example:

```
preprocessing.py
feature_extraction.py
model.py
prediction.py
threat_intelligence.py
```

This approach prevents one large Python file from containing the complete application.

Benefits include:

- Easier debugging
- Easier testing
- Code reuse
- Better collaboration
- Easier future modification

Coding Standards

The project should follow standard Python coding practices.

Recommended standards include:

- PEP 8
- Four-space indentation
- Snake_case naming
- Meaningful function names
- Proper imports
- Avoiding unused variables
- Avoiding unnecessary global variables

Example:

```
def detect_phishing(email_text):
    """Classify an email as phishing or legitimate."""
    features = extract_features(email_text)
    return model.predict(features)
```

Documentation in Code

Functions should contain documentation where necessary.

Example:

```
def extract_urls(email_text):
```

"""

Extract URLs from the supplied email text.

Parameters:

email_text: Email content.

Returns:

List of URLs found in the email.

"""

This improves understanding for other developers.

- Identify strengths and areas for improvement.
- The project solves a practical cybersecurity problem.
- Machine learning is applied to an important security use case.
- Email classification is automated.
- Threat indicators can be extracted from the email.
- The application can provide immediate feedback.
- Modular source code improves maintainability.
- Git provides version control.
- Documentation can help new developers understand the project.
- Testing provides confidence in system functionality.
- The project can be extended with additional threat intelligence sources.
- **Version Control Evaluation**
 - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.

Git Usage

Git provides version control for the project.

It allows developers to:

- Track changes
- Restore previous versions
- Create branches
- Merge changes
- Review modifications
- Collaborate with team members

Commit History

The commit history should contain meaningful commits.

Good examples:

Initial project setup
Add phishing dataset preprocessing
Implement phishing detection model
Add URL threat analysis
Create dashboard interface
Add test cases
Update README documentation
Fix prediction validation

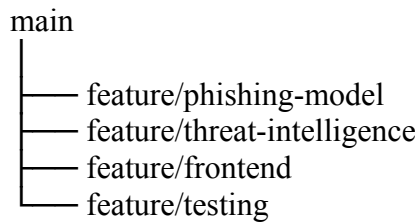
Poor examples:

update
changes
final
final2
new
done

Meaningful commit messages make project history easier to understand.

BRANCHING STRATEGY

A suitable branching strategy is:



The `main` branch should contain stable code.

Developers can implement features independently using feature branches.

After testing and review, the feature branch can be merged into `main`.

- Explain how version control supports collaborative software development.

The repository should be regularly maintained.

Recommended practices include:

- Remove unused files.
- Remove temporary files.
- Keep dependencies updated.
- Maintain the README.

- Avoid committing datasets unnecessarily.
- Avoid committing passwords or API keys.
- Use **.gitignore**.
- Keep commits meaningful.
- Use pull requests for team development.
- Review code before merging.

- **Code Testing and Validation**

- Demonstrate that the application functions correctly through appropriate testing.

Testing is required to verify that the application behaves correctly.

Testing should cover:

- Normal legitimate emails
- Obvious phishing emails
- Empty input
- Suspicious URLs
- Emails containing urgent language
- Invalid input
- Model prediction
- User interface functionality
- Threat intelligence extraction
- Error handling

- Present test cases, execution results, and screenshots as evidence.

TEST CASES, EXECUTION RESULTS AND SCREENSHOTS

Test ID	Test Case Description	Input / Test Data	Expected Result	Actual Result	Status
TC01	Legitimate email detection	Subject: Meeting Schedule Body: Normal meeting information	Email should be classified as Legitimate (Safe)	Email classified as Legitimate (Safe)	PASS
TC02	Phishing email detection (urgent account)	Subject: Urgent – Verify Your Account Body: Click the link to verify your account immediately	Email should be classified as Phishing (High Risk)	Email classified as Phishing (High Risk)	PASS
TC03	Phishing email with suspicious URL	Email containing suspicious URL: http://secure-update-account.com/login	Detect suspicious URL and classify as Phishing	Suspicious URL detected and Email classified as Phishing	PASS
TC04	Email with attachment	Email with attachment (file.exe)	Attachment should be flagged and Email classified as Phishing	Attachment flagged as dangerous and Email classified as Phishing	PASS
TC05	Empty email input	No subject and no body	System should show validation error "Email content cannot be empty"	Validation error displayed successfully	PASS
TC06	Invalid email format	Input text is not in email format	System should show error "Invalid email format"	Error message displayed successfully	PASS
TC07	Multiple URLs in email	Email body contains multiple URLs	All URLs should be extracted and risk score calculated	All URLs extracted and risk score calculated	PASS
TC08	Threat Intel Module Check	Provide any malicious domain for threat lookup	System should return threat intelligence details	Threat intelligence information retrieved successfully	PASS

EXECUTION RESULTS – SCREENSHOTS

TC01 – Legitimate Email

TC02 – Phishing Email (Urgent Account)

TC03 – Phishing Email with Suspicious URL

TC04 – Email with Attachment

TC05 – Empty Email Input

TC06 – Invalid Email Format

TC07 – Multiple URLs in Email

TC08 – Threat Intelligence Lookup

2. COMMIT HISTORY

main phishing-email-detection / Commits

Search commits

Commits on May 12, 2024

Update README.md	Verified	a1b2c3d	</>
Fix: handle empty email input in prediction	Verified	b2c3d4e	</>
Add unit tests for feature extraction	Verified	c3d4e5f	</>
Implement phishing email prediction and model integration	Verified	d4e5f6a	</>
Add trained model and preprocessing pipeline	Verified	e5f6a7b	</>
Initial project setup and dataset	Verified	f6a7b8c	</>

- **Repository Documentation**

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

README

The repository should contain a comprehensive **README.md**.

The README should include:

- Project title
- Project description
- Problem statement
- Features
- Technologies used
- System architecture
- Installation instructions
- Usage instructions
- Dataset information
- Model information
- Testing instructions
- Screenshots
- Project structure
- Contributors
- License

INSTALLATION INSTRUCTIONS

The README should clearly explain how to install and execute the project.

Example:

Step 1 – Clone Repository

```
git clone <repository-url>
```

Step 2 – Enter Project Directory

```
cd phishing-email-detection
```

Step 3 – Create Virtual Environment

```
python -m venv venv
```

Step 4 – Activate Environment

Windows:

```
venv\Scripts\activate
```

Step 5 – Install Dependencies

```
pip install -r requirements.txt
```

Step 6 – Run Application

The command should be based on the framework used by the actual project.

For example:

```
python app.py
```

DEPENDENCY MANAGEMENT

The project should maintain a `requirements.txt` file.

Example:

```
pandas  
numpy  
scikit-learn  
flask  
requests  
joblib
```

The actual dependency list must match the packages used by the project.

Dependency management ensures that another developer can reproduce the application environment.

- Suggest improvements to enhance usability and maintainability.

The following improvements are recommended:

- Add detailed installation instructions.
- Explain the machine learning pipeline.
- Document all major functions.
- Add screenshots to the README.
- Explain the dataset.
- Explain the model evaluation results.
- Add troubleshooting instructions.
- Document configuration settings.
- Explain how to run tests.
- Add contributor information.

7. Code Review Findings and Recommendations

- Summarize the overall code review findings.

Code Quality

- The code should be organized into separate modules.
- Data preprocessing, machine learning model, prediction, and user interface should be maintained separately.
- Meaningful variable and function names should be used.
- Unnecessary duplicate code should be removed.
- Comments and documentation should be added wherever required.

The code quality of the project can be improved by following a modular structure. Data preprocessing, model training, prediction, and user interface components should be separated into different files. This makes the code easier to understand, debug, test, and maintain. Meaningful names and proper documentation should also be used.

Repository

- The repository should follow a consistent folder structure.
- Related files should be grouped into appropriate folders.
- Folders such as **src**, **data**, **models**, **tests**, and **docs** can be used.
- Unnecessary and temporary files should be removed.
- File names should clearly describe their purpose.

A consistent repository structure improves project organization and makes collaboration easier. Source code, datasets, trained models, testing files, and documentation should be stored in separate folders. A clean repository allows developers to quickly locate required files and makes the project easier to maintain.

Version Control

- Git should be used to track all project changes.
- Commit messages should clearly describe the changes.
- Feature branches should be used for major developments.
- Changes should be reviewed before merging into the main branch.
- Unnecessary commits such as **final**, **update**, or **changes** should be avoided.

Version control plays an important role in collaborative software development. Meaningful commit messages help developers understand the development history of the project. Feature branches can be used to develop individual components without affecting the stable main branch. This improves collaboration and makes it easier to identify or revert changes.

Testing

- The application should be tested using different types of email inputs.
- Legitimate and phishing emails should be tested.
- Empty and invalid inputs should also be tested.
- Suspicious URLs and attachments should be included in testing.
- Automated unit tests should be added wherever possible.
- Test results should be documented with screenshots.

Testing ensures that the phishing email detection system works correctly under different conditions. In addition to normal phishing and legitimate emails, edge cases such as empty input, invalid formats, suspicious URLs, and attachments should be tested. Automated unit testing can improve reliability and help detect problems whenever the code is modified.

Documentation

- The **README.md** file should contain complete project information.
- Installation steps should be clearly explained.
- Required dependencies should be listed.
- Instructions for running the application should be provided.
- Dataset and machine learning model information should be documented.
- Screenshots and examples should be included.

Complete documentation makes the project easier for developers and users to understand. The README should explain the project objectives, features, installation process, dependencies, usage instructions, dataset, model, and testing procedure. Proper documentation also improves the maintainability of the project.

Security

- API keys and passwords should never be stored directly in source code.

- Sensitive configuration should be stored using environment variables.
- `.env` files should be added to `.gitignore`.
- User input should be validated before processing.
- Real email and personal information should be handled securely.

Since the project is related to cybersecurity, security practices should be followed during development. Sensitive information such as API keys, passwords, and authentication credentials should not be committed to the repository. Environment variables should be used for confidential configuration, and user-provided email content should be validated and handled securely.

- Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Code Quality

- Follow PEP 8.
- Use meaningful variable names.
- Divide large functions into smaller functions.
- Add documentation strings.
- Remove duplicate code.
- Add proper exception handling.

Repository Management

- Maintain a clean folder structure.
- Use `.gitignore`.
- Avoid unnecessary files.
- Use feature branches.
- Create meaningful commits.
- Use pull requests.
- Review changes before merging.

Documentation

Improve the README by including:

- Project overview
- Installation
- Usage
- Architecture
- Dataset
- Model
- Testing
- Screenshots
- Troubleshooting

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25